

Biody Manager API — Technical Integration Description

Prepared by: Lifeline Health ehf. **Audience:** Medalia engineering (leads + implementing engineer / coding agent). **Purpose:** A complete, self-contained technical description of the Biody Manager API connection, so the Medalia team can integrate body-composition measurements into their own systems. **Status:** Distilled from Lifeline Health's production integration. Every wire shape and quirk below was verified against Biody's live API. The "gotchas" in particular are hard-won and are **not** in Biody's official documentation.

Note on secrets. This document deliberately contains **no live credentials** — service-account email/password, the HMAC signing secret, and project keys are delivered separately through a secure channel. Everything else needed to implement is here.

Part I — Executive summary

What is Biody?

Biody Manager (<https://www.biodymanager.com>) is the cloud platform behind the **Biody Xpert / BX3** body-composition analysers — the impedance-based scales used by clinicians and gyms to capture weight, fat mass, muscle mass, total body water, phase angle, BMR, etc.

When a patient steps on the scale, the device uploads the measurement to Biody Manager. The Manager exposes a REST API (Symphony-based, JWT-authenticated) that lets a partner application:

1. **Create patients** — link a Medalia user to a Biody patient record.
2. **Group patients** — for B2B / multi-tenant deployments, each customer can be its own folder.
3. **Pull measurements** — receive each new analysed measurement and store it in your own database alongside the patient record.

There is no public Biody portal for end-users — the partner app *is* the patient-facing experience.

Why integrate

Lifeline Health integrated Biody using this exact pattern in 2025/2026 and shipped it to production. The integration replaces the standalone Biody clinician portal that patients otherwise log into separately. After the integration:

- Patients see body-composition history inside the app — no second login.
- Trends are correlated against everything else the app tracks (mood, sleep, exercise, blood markers).
- Clinicians get one chart instead of jumping between systems.
- The Biody scale operator (nurse / coach) needs no changes — they keep using the BX3 hardware exactly as today.

Effort estimate

Lifeline's integration took roughly **three engineer-weeks** end-to-end including the gotchas. With this guide a fresh team should land in **5-10 working days**:

- 1-2 days: auth flow, token caching, first patient created against a Biody sandbox account.

- 1–2 days: notification poll + measurement ingestion into your DB schema.
- 2–3 days: patient-group / company affiliation if you have B2B tenancy.
- 1–2 days: webhook *or* cron-driven reconciliation, error retries, monitoring.
- 1 day: secret rotation, HMAC signing between your backend and worker, documentation.

What you need from Biody up front

A short email to your Biody account manager (template in §11). Ask for:

1. **A partner service account** — email + password used by your backend to authenticate (`/api/login_check`). Single shared credential per partner — not one-per-user. Get a non-personal email (e.g. `api@yourcompany.com`).
2. **The BM-Application header value** — Lifeline uses `PartnerApp` . Confirm yours.
3. **The measurement template id** for the device(s) you'll receive data from. Lifeline uses `94586` (BX3 Dashboard). Different device families use different ids.
4. **Confirmation of the physical-activity enum ids** — Biody has 5 levels (`very low` / `low` / `average` / `strong` / `very high`). The integer ids 1–5 are inferred but should be verified per environment.
5. **A test patient + test measurement** in the sandbox so you can ingest one round-trip before going live.

Recommended architecture (one paragraph)

A small backend worker (Lifeline's runs as a Supabase Edge Function; any serverless function or persistent service works) holds the partner credential, caches the JWT token in a key/value table, and exposes three internal endpoints to the rest of your app: `create-patient` , `poll` (cron-driven, every 3 minutes), and `reconcile` (cron-driven, daily, belt-and-braces). The app never talks to Biody directly — it asks the worker, which talks to Biody on its behalf. This keeps the credential out of the app bundle and lets you swap providers later without touching client code.

Part II — Technical integration guide

0. Conventions used in this doc

- All snippets are TypeScript / Deno (Supabase Edge Function flavour). Translate to your stack as needed; the wire shapes don't change.
- `${BASE}` = Biody Manager base URL, default `https://www.biodymanager.com` .
- `${LOCALE}` = path-segment locale, one of `en` / `fr` / etc. Lifeline uses `en` even though the UI is Icelandic — locale only affects API response language.
- `${TOKEN}` = JWT obtained from `/api/login_check` .
- All requests must carry the header `BM-Application: PartnerApp` — **this is undocumented but mandatory**. Without it Biody returns a generic 400.

1. Architecture overview

Run a small worker (Edge Function, Lambda, container — any HTTPS-reachable service) that owns the Biody partner credential. The app never sees the credential or the JWT.

```

[App] → POST /your-worker/create-patient → [Worker] → POST /api/patients → [Biody]
[Cron] → POST /your-worker/poll → [Worker] → GET /api/notifications → [Biody]
[Cron] → POST /your-worker/reconcile → [Worker] → GET /api/measurements → [Biody]
                                         ↓
                                         → INSERT/UPSERT measurements into your DB

```

Why a worker, not direct calls from the app:

- Credential stays server-side. App-bundle compromise doesn't leak the partner login.
- Token caching is centralised — one JWT shared across requests instead of one per device.
- Rate limits / retries / observability live in one place.
- Trivial to swap to another provider (Polar Verity Sense, Tanita, etc.) later.

1.1 Worker → app server auth

If the worker is hit by your application backend (typical), require an HMAC signature on every request rather than a long-lived bearer token. Lifeline's pattern:

```

// On the calling side (app server):
const ts = Math.floor(Date.now() / 1000).toString();
const mac = createHmac("sha256", SIGNING_SECRET).update(`${ts}.${bodyText}`).digest("hex");
headers["X-Lifeline-Signature"] = `t=${ts},v1=${mac}`;

// On the worker side:
function verifySignature(req: Request, bodyText: string): boolean {
  const sig = req.headers.get("X-Lifeline-Signature") || "";
  const m = /^t=(\d+),v1=([0-9a-f]+)$/i.exec(sig);
  if (!m) return false;
  const ts = parseInt(m[1], 10);
  if (Math.abs(Date.now()/1000 - ts) > 300) return false; // 5-min freshness
  const expected = m[2];
  // HMAC-SHA-256 the same `${ts}.${bodyText}` and compare in constant time.
  // If anything mismatches → 401.
}

```

Refuse fallback to a static service-role token for production — a single leaked key replays forever.

2. Authentication

2.1 Login

Single endpoint, single shared service account.

```

POST ${BASE}/api/login_check
Headers:
  Content-Type: application/json
  BM-Application: PartnerApp
Body:
  { "email": "<service-account-email>", "password": "<service-account-password>" }

```

Response:

```
{
  "token": "<JWT>",
  "user": {
    "id": 1234,
    "email": "...",
    "tokenTtl": 3600,           // seconds; not always present, default to 3600
    "...": "..."
  }
}
```

Failure on bad credentials: **401** with body like `{"code":401,"message":"Invalid credentials."}`. **One specific gotcha: if the service account is flagged as "must change password on next login" inside Biody Manager, login returns **401** {"message":"Change_Your_Password"}. Biody admins or password-reset flows can put the account in this state silently. Resolution: log into the Biody web UI with the same credentials, complete the forced password-change form, then update your secret.**

2.2 Token caching

Cache the token in your DB or KV store. Lifeline's table:

```
CREATE TABLE biody_auth_cache (
  id          INT PRIMARY KEY DEFAULT 1,          -- always 1, single-row
  token       TEXT NOT NULL,
  expires_at  TIMESTAMPTZ NOT NULL,
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Lookup pattern:

```
async function getToken(): Promise<string> {
  const cached = await db.query("SELECT token, expires_at FROM biody_auth_cache WHERE id = 1");
  // Re-use the cached token if it has more than 60 seconds left.
  if (cached?.token && new Date(cached.expires_at) > new Date(Date.now() + 60_000)) {
    return cached.token;
  }
  // Otherwise log in afresh.
  const res = await fetch(`${BASE}/api/login_check`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "BM-Application": "PartnerApp" },
    body: JSON.stringify({ email: BIODY_EMAIL, password: BIODY_PASSWORD }),
  });
  if (!res.ok) throw new Error(`Biody login failed: ${res.status} ${await res.text()}`);
  const json = await res.json();
  const ttl = json.user?.tokenTtl ?? 3600;
  const expiresAt = new Date(Date.now() + ttl * 1000);
  await db.query(
    "INSERT INTO biody_auth_cache (id, token, expires_at) VALUES (1, $1, $2) ON CONFLICT (id) DO UPDATE SET token = $3, expires_at = $4",
    [json.token, expiresAt, json.token, expiresAt],
  );
  return json.token;
}
```

2.3 401 retry

Even within the cached TTL, Biody occasionally returns `401` (token revoked server-side, deploy on their end, etc). Wrap every Biody call in a retry that invalidates the cache and tries once:

```
async function biody<T>(path: string, init: RequestInit = {}, retry = true): Promise<T> {
  const token = await getToken();
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
      "Content-Type": "application/json",
      "BM-Application": "PartnerApp",
      "Authorization": `Bearer ${token}`,
      ...(init.headers || {}),
    },
  });
  if (res.status === 401 && retry) {
    await db.query("DELETE FROM biody_auth_cache WHERE id = 1");
    return biody<T>(path, init, false);
  }
  if (!res.ok) throw new Error(`Biody ${init.method || "GET"} ${path} → ${res.status} ${await res.text()}`);
  return res.json() as Promise<T>;
}
```

Don't retry on non-401 errors — Biody's 5xxs are usually upstream of you and retrying compounds load.

3. Patients

3.1 Create

```
POST ${BASE}/${LOCALE}/api/patients?id=0
```

The `?id=0` query parameter is required — Biody uses 0 to mean "new resource". Without it the request 404s.

Required fields (everything below):

Field	Type	Notes
<code>first_name</code>	string	
<code>last_name</code>	string	
<code>email</code>	string	optional but strongly recommended
<code>mobile_number</code>	string	optional
<code>birth_date</code>	ISO 8601 with time	must include <code>T00:00:00.000Z</code> suffix — date-only <code>"2000-01-10"</code> is rejected
<code>gender</code>	<code>"Male"</code> or <code>"Female"</code>	exact casing matters
<code>height</code>	string	two-decimal string e.g. <code>"170.00"</code> — number values rejected
<code>cup_size</code>	string	mandatory even for males; <code>"N/A"</code> is accepted
<code>physical_activity</code>	integer	bare integer (1-5 in our env), not <code>{ id: 1 }</code>
<code>template</code>	integer	the measurement template id, e.g. <code>94586</code>
<code>patient_groups</code>	<code>number[]</code>	optional, see §4

Example payload:

```
{
  first_name: "Jón",
  last_name: "Jónsson",
  email: "jon@example.is",
  birth_date: "1985-04-12T00:00:00.000Z",
  gender: "Male",
  height: "182.00",
  cup_size: "N/A",
  mobile_number: "+3548991234",
  physical_activity: 3,           // "average"
  template: 94586,               // BX3 Dashboard
  patient_groups: [42],         // optional
}
```

Successful response includes the new patient's id; uuid is sometimes omitted on create. Pattern:

```
const created = await biody<any>(`/${LOCALE}/api/patients?id=0`, {
  method: "POST",
  body: JSON.stringify(payload),
});
const id = created?.id ?? created?.patient?.id ?? created?.data?.id;
let uuid = created?.uuid ?? created?.patient?.uuid;
if (id && !uuid) {
  // Follow-up GET to pick up the uuid Biody assigns server-side.
  const detail = await biody<any>(`/${LOCALE}/api/patients/${id}`);
  uuid = detail?.uuid ?? detail?.patient?.uuid ?? detail?.data?.uuid;
}
```

Persist **both** **id** and **uuid** on your patient row — different Biody endpoints key by different identifiers (notifications use id, some webhook payloads use uuid).

3.2 Idempotency

Always check whether your patient already has a **biody_patient_id** before calling create — Biody won't dedupe on email. Lifeline's pattern:

```
if (client.biody_patient_id) {
  return { existing: true, biody_patient_id: client.biody_patient_id, biody_uuid: client.biody_uuid };
}
// else: create + persist
```

If you allow concurrent activations (e.g. two browser tabs), wrap with a DB-side claim so two requests don't both create:

```
-- claim_biody_activation: returns true if the caller wins the claim,
-- false if another request already holds it.
CREATE OR REPLACE FUNCTION claim_biody_activation(p_client_id UUID)
RETURNS BOOLEAN LANGUAGE plpgsql AS $$
BEGIN
    UPDATE clients
    SET biody_activation_started_at = now()
    WHERE id = p_client_id
        AND biody_patient_id IS NULL
        AND (biody_activation_started_at IS NULL
            OR biody_activation_started_at < now() - INTERVAL '2 minutes');
    RETURN FOUND;
END $$;
```

3.3 Looking up by email

There is no documented "find patient by email" endpoint we found reliable. Always create-or-reuse from your own DB; don't rely on Biody to dedupe.

4. Patient groups (multi-tenant / B2B)

If Medalia has B2B customers (one company, many employees), use Biody patient groups to keep each company's patients separate.

4.1 Create or find a group

```
GET  ${BASE}/${LOCALE}/api/patientgroups?limit=200&order_by[patientGroup.name]=asc
POST ${BASE}/${LOCALE}/api/patientgroups?id=0
    body: { name, color, activated: true, share_level: "private" }
```

Gotcha: the `name` column in Biody's DB is a `varchar` capped at ~20 characters. Sending a longer name returns a generic 500. Truncate:

```
const safeName = (company.name || "").slice(0, 20).trim() || company.id.slice(0, 8);
```

`color` should be a hex string (`#3B82F6` etc). `share_level` accepts `private` | `partner` | `public` — `private` is correct for tenant isolation.

4.2 Attach a patient

Pass the group id(s) on patient creation:

```
payload.patient_groups = [groupId];
```

Lifeline persists `companies.biody_group_id` so we don't re-create the group on every employee onboarding.

5. Measurements

5.1 The model

A patient has many measurements. Each measurement has:

- `id` — integer
- `uuid` — string (sometimes only available on the detail endpoint)
- `patient` — `{ id, uuid }` or just integer
- `msrDate` / `created_at` — ISO timestamp
- `status` — `"Analyzed"` (final), or earlier states like `"Pending"` while the device is still uploading. **Only ingest `Analyzed`.**
- ~50 numeric fields: `weight`, `height`, `fm_kg`, `fm_pc`, `fm_hc_pc`, `mms_kg`, `mmhi`, `dffm_kg`, `phase_angle`, `bmr`, `tbw`, `ecw_pc`, `icw_pc`, `asmhi`, `whtr`, `waist_size`, `hips_size`, etc.

The full list and their meaning is in Biody's hardware manual; ask your Biody contact for the data dictionary if needed.

5.2 Notification-driven poll

Biody surfaces "new measurement available" via the notifications endpoint:

```
GET ${BASE}/${LOCALE}/api/notifications?filters[notification.controller]=measurement&filters[notification.read]=false
```

Each unread notification has a `primary_key` field equal to the measurement id. Pull the measurement, ingest, then mark read:

```
PUT ${BASE}/${LOCALE}/api/notifications/${notification.id}
body: { read: true }
```

Run this on a 3-minute cron (Lifeline). Faster than 1-minute is unnecessary; slower than 10-minute is user-visible lag.

5.3 Pulling a single measurement

Prefer the detail endpoint over the list filtered by id — the list endpoint's `filters[measurement.id]=...` is unreliable in our experience.

```
const detail = await biody<any>(`/${LOCALE}/api/measurements/${id}`);
const m = detail?.results?.[0] ?? detail?.data ?? detail; // Biody isn't consistent across endpoints; defend.
```

If `m.status !== "Analyzed"` you may want to trigger interpretation manually:

```
POST ${BASE}/api/interpret?controller=measurements&id=${m.id}
```

But ingest whatever you have anyway — most fields are populated even pre-Analyzed.

5.4 Daily reconcile

The notification feed occasionally drops events (we've seen it). Run a reconciliation sweep once a day:

```
const linked = await db.query("SELECT id, biody_patient_id FROM patients WHERE biody_patient_id IS NOT NULL");
for (const p of linked) {
  const list = await biody<any>(
    `/${LOCALE}/api/measurements?filters[measurement.patient]=${p.biody_patient_id}&filters[measurement.status]=Analy
  );
  for (const m of (list.results || list.data || list)) {
    await ingestMeasurement(m, p.id); // upsert by Biody measurement id
  }
}
```

Upsert by `biody_measurement_id` so re-ingesting the same measurement is a no-op.

5.5 Field mapping

Lifeline's storage schema (rename to taste):

Your column	Biody field	Notes
<code>biody_measurement_id</code>	<code>id</code>	unique key
<code>measured_at</code>	<code>msrDate</code> ?? <code>created_at</code>	
<code>weight_kg</code>	<code>weight</code>	
<code>height_cm</code>	<code>height</code>	
<code>body_fat_pct</code>	<code>fm_hc_pc</code> ?? <code>fm_pc</code>	use heuristic-corrected if present
<code>fat_mass_kg</code>	<code>fm_kg</code>	
<code>muscle_mass_kg</code>	<code>mms_kg</code>	
<code>muscle_mass_pct</code>	computed	<code>mms_kg / weight * 100</code>
<code>phase_angle</code>	<code>phase_angle</code>	
<code>bmr_kcal</code>	<code>bmr</code>	round to integer
<code>tbw_l</code>	<code>tbw</code>	total body water
<code>ecw_pct</code>	<code>ecw_pc</code>	extracellular water %
<code>icw_pct</code>	<code>icw_pc</code>	intracellular water %
<code>bone_mineral_idx</code>	<code>asmhi</code>	
<code>visceral_fat_idx</code>	<code>whtr</code>	
<code>waist_cm</code>	<code>waist_size</code>	
<code>hips_cm</code>	<code>hips_size</code>	
<code>raw</code> (jsonb)	full payload	keep the full object so you can derive new fields later without re-pulling

5.6 Measurement quality gates

Reject (or flag) physiologically impossible values to avoid sending nonsense to a patient's chart:

```
if (weight !== null && (weight < 20 || weight > 300))    flag.push("weight_out_of_range");
if (height !== null && (height < 100 || height > 250))   flag.push("height_out_of_range");
if (bodyFat !== null && (bodyFat < 2 || bodyFat > 70))    flag.push("body_fat_out_of_range");
if (phaseAngle !== null && (phaseAngle < 1 || phaseAngle > 15)) flag.push("phase_angle_out_of_range");
```

We *ingest* anyway (so the device reading isn't lost) but mark for review.

6. Activity-level mapping

Biody's `physical_activity` enum has 5 levels with these labels:

Biody label	Likely id (verify in your env)
very low	1
low	2
average	3
strong	4
very high	5

Map your app's activity scale onto these. Lifeline's mapping (sedentary → very low, etc):

```
const ACTIVITY_MAP: Record<string, number> = {
  sedentary: 1,
  light: 2,
  moderate: 3,
  very_active: 4,
  extra_active: 5,
};
```

Always make this configurable (env var or DB row) so when Biody reorders ids in a release you don't need a deploy.

7. The 8 gotchas Lifeline hit

In rough order of pain inflicted:

1. **BM-Application: PartnerApp** header is mandatory but undocumented. Without it, every endpoint 400s with a generic message.
2. **birth_date** must be ISO 8601 *with time*. "2000-01-10" is rejected; send "2000-01-10T00:00:00.000Z".
3. **height** is a string with two decimals. 170 and 170.0 both fail; "170.00" works.
4. **cup_size** is required even for male patients. "N/A" is accepted.
5. **physical_activity** is a bare integer. { "id": 3 } fails; 3 works.
6. **Patient-group name** is capped at ~20 chars by an underlying varchar. Truncate before POST.
7. **Listing measurements by id is unreliable**. Use the single-resource detail endpoint (/measurements/{id}) when you have an id.
8. **Token can be poisoned by a forced password change**. Login returns 401 {"message": "Change_Your_Password"} — see §2.1. Resolution requires a manual password change in the Biody web UI.

A second-tier list (encountered, less painful):

- The create-patient response sometimes omits uuid ; follow up with GET /patients/{id} .
- Some endpoints return { data: [...] } , others { results: [...] } , others a bare array. Defend with (x.results || x.data || x) .
- Notifications occasionally drop events under heavy load — run a daily reconcile sweep as a safety net.

8. Observability

Log every Biody call with path , method , status , duration_ms , cache_hit (was the JWT cached or freshly minted), and outcome . Lifeline writes a per-run summary:

```
CREATE TABLE biody_sync_runs (
  id          BIGSERIAL PRIMARY KEY,
  mode        TEXT NOT NULL,          -- 'create_patient' | 'notifications' | 'reconcile'
  processed   INT  NOT NULL DEFAULT 0,
  errors      INT  NOT NULL DEFAULT 0,
  last_error  TEXT,
  duration_ms INT,
  ran_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Send Biody-side errors to Sentry (or your equivalent) with the request payload and response body. The first six months of integration almost always require post-hoc payload inspection to figure out *which* of the eight gotchas above bit you.

9. Secrets

Single-source secrets:

Name	Where it lives	Rotation cadence
BIODY_EMAIL	worker env	rare
BIODY_PASSWORD	worker env	quarterly + on suspected leak
BIODY_LOCALE	worker env ("en" for most)	never
BIODY_TEMPLATE_ID	worker env or DB	per device family
BIODY_ACTIVITY_IDS	env override (e.g. "sedentary:1,light:2,...")	as needed
WORKER_HMAC_SECRET	shared between worker + app server	quarterly

Never put any of these in a client bundle.

10. Suggested testing path

- Day 1.** Hit `/api/login_check` from your worker. Cache the token. Verify 401 retry works by manually expiring the cache row.
- Day 2.** Create one test patient. Verify `id` and `uuid` are persisted. Re-run — should reuse, not create a duplicate.
- Day 3.** Manually take a measurement on a Biody device against the test patient. Run the poll loop. Verify the measurement lands in your DB with all expected fields, status `Analyzed`, and notification marked read.
- Day 4.** Add a patient group (if multi-tenant) and verify the second test patient is created inside it.
- Day 5.** Wire the cron schedule (poll every 3 min, reconcile daily). Smoke-test for 24h before rolling to real users.

11. What to ask Biody up front

A short email to your Biody account contact saves a week:

Subject: Medalia partner API integration – onboarding questions

Hi <name>,

We're building a partner integration with Biody Manager so our patients see their measurements directly inside Medalia. Could you confirm the following for our environment so we can scope correctly:

1. Service-account email + initial password (to be rotated on first login).
2. The exact value to use for the BM-Application header.
3. The measurement template id(s) for the BX3 family our partners use.
4. The numeric ids for the physical_activity enum (very low / low / average / strong / very high – id 1..5?).
5. Whether webhook delivery for measurements is available, or if we should poll /api/notifications.
6. A test patient + at least one measurement on the sandbox so we can smoke-test before going to production.
7. Rate limits on /api/login_check, /api/patients, /api/measurements, /api/notifications. We plan to poll notifications every 3 minutes and reconcile daily.

Thanks – we'll send a target go-live date once we have items 1–6.

12. Reference implementation

Lifeline Health's worker (Supabase Edge Function, ~880 lines of TypeScript) implements every code path described above. It can be shared with Medalia's engineering team on request as a starting reference — let your Lifeline counterpart know if you'd like the source.

13. Endpoint quick reference

All paths relative to `${BASE}` = `https://www.biodymanager.com`. Every request carries `BM-Application: PartnerApp`; all except login also carry `Authorization: Bearer ${TOKEN}`.

Method	Path	Purpose
POST	/api/login_check	Obtain JWT (body: {email, password})
POST	/\${LOCALE}/api/patients?id=0	Create patient
GET	/\${LOCALE}/api/patients/{id}	Fetch patient (uuid backfill)
GET	/\${LOCALE}/api/patientgroups?limit=200&order_by[patientGroup.name]=asc	List groups
POST	/\${LOCALE}/api/patientgroups?id=0	Create group
GET	/\${LOCALE}/api/notifications? filters[notification.controller]=measurement&filters[notification.read]=false&limit=100	Unread measurement notifications
PUT	/\${LOCALE}/api/notifications/{id}	Mark notification read (body: {read:true})
GET	/\${LOCALE}/api/measurements/{id}	Fetch single measurement (preferred)
GET	/\${LOCALE}/api/measurements?filters[measurement.patient]= {pid}&filters[measurement.status]=Analyzed&order_by[measurement.msrDate]=desc&limit=20	List a patient's measurements (reconcile)
POST	/api/interpret?controller=measurements&id={id}	Trigger interpretation

Prepared by Lifeline Health ehf. Questions on anything in this document — contact Mads at Lifeline Health.